

CloudPulse – Study Notes

Everything about the project, explained in simple English, with all the technical words

Student: Cecil Baraik • Roll 2025-BSCIT/004A

Project: CloudPulse – Cloud Resource Monitoring Dashboard

Technologies: Java 21, Spring Boot 3.5, MySQL 8, React 19, WebSocket, JWT, OSHI, Gmail SMTP, OpenPDF

How to read: chapters 1–3 build the basics, 4–14 explain each part of the project, 15–18 cover tools, running, problems and future work, 19 is an A–Z glossary and 20 is a one-page revision list for the night before the viva. Blue boxes are examples, orange boxes are important notes, green boxes are key summaries.

Contents

Chapter 1	The big picture
Chapter 2	Basic concepts: cloud and monitoring
Chapter 3	How web applications work
Chapter 4	Java and Spring Boot (the backend)
Chapter 5	Project structure: every file explained
Chapter 6	The database
Chapter 7	Live updates: the 3-second loop and WebSocket
Chapter 8	Monitoring: simulated servers and your real PC
Chapter 9	Alerts and email notifications
Chapter 10	Costs and budget
Chapter 11	Logs
Chapter 12	Reports and export
Chapter 13	Security
Chapter 14	The frontend (React)
Chapter 15	Tools: IntelliJ, Git and GitHub
Chapter 16	How to run the project
Chapter 17	Problems faced and how they were solved
Chapter 18	Limitations and future scope
Chapter 19	Glossary A–Z
Chapter 20	Quick revision: 25 points to remember

The big picture

1.1 What is CloudPulse?

CloudPulse is a **web application** that watches servers and shows their health on one live screen. Every 3 seconds it shows how busy each server is (CPU), how much memory it uses, how full its disk is and how much network data it sends and receives.

When something goes wrong (for example CPU above 95%), CloudPulse raises an **alert**, shows a pop-up, writes it in the logs and, if it is serious, sends an **email**. It also shows what the servers **cost**, shows live **logs**, lets an admin **start and stop** servers, and exports **PDF and CSV reports**.

Real-life example: Think of a hospital patient monitor. It shows heart rate, blood pressure and oxygen all the time and beeps when something is dangerous. CloudPulse is the same thing, but for computers. That is why the logo is a heartbeat line.

1.2 The problem it solves

- **Many consoles:** companies use servers on AWS, Azure and Google Cloud. Each has its own website, so you must open many tabs.
- **Problems found too late:** people check by refreshing pages, so failures are noticed only when users complain.
- **Hidden cost:** cloud servers are paid per hour. A forgotten server keeps costing money.

CloudPulse answers four questions on one screen: **Is everything healthy? What happened? What is about to fail? What does it cost?**

1.3 What is in the project (the 11 resources)

The project monitors **11 resources**. Ten are **simulated** cloud resources (because real AWS/Azure accounts cost money) and one is **real**: your own PC.

Name	Type	Provider	Price per hour
web-server-01, web-server-02	VM	AWS	\$0.0448 each
api-gateway-prod	Container	AWS	\$0.0494
postgres-main	Database	AWS	\$0.152
media-storage	Storage	AWS	\$0.031
analytics-vm	VM	GCP	\$0.155
image-resizer	Function	GCP	\$0.006
redis-cache	Database	Azure	\$0.10

ml-worker-gpu	VM (with GPU)	Azure	\$3.06
backup-server	VM (starts stopped)	GCP	\$0.017
Your PC "(this PC)"	LOCAL_HOST	LOCAL	free

These are created automatically the first time the app starts, by a class called `DataSeeder`. The backup-server starts as STOPPED so you can show the Start button in the demo. The GPU server is the most expensive, so the cost page suggests running it only in working hours.

1.4 The whole project in one paragraph

The **React** website in the browser logs in to the **Spring Boot** server and gets a **JWT token**. Every 3 seconds a **scheduled job** on the server gets new values (from a **simulation** or from **OSHI** for the real PC), saves them in **MySQL**, checks the **alert rules**, writes **logs**, and pushes everything to all open browsers through a **WebSocket**. The browser redraws the charts without refreshing. Critical alerts are also **emailed** through **Gmail**.

1.5 Demo logins

Username	Password	Role	Can do
admin	admin123	ADMIN	Everything
viewer	viewer123	VIEWER	Only look, acknowledge alerts, export

Basic concepts: cloud and monitoring

2.1 Server

A **server** is a computer whose job is to serve other computers, for example to send web pages, store data or run a program. It usually has no screen and runs all day.

2.2 Cloud computing

Cloud computing means renting computers, storage and services over the internet instead of buying your own hardware. You pay only for what you use, usually per hour or per second.

- **AWS** (Amazon Web Services), **Azure** (Microsoft) and **GCP** (Google Cloud Platform) are the three biggest cloud providers.
- **Region:** the city/area where the data centre is, for example `ap-south-1` = Mumbai (AWS), `centralindia` = Pune (Azure).
- **Instance type:** the size of the rented computer, for example `t3.medium` = 2 CPUs and 4 GB RAM on AWS.

2.3 Types of cloud resources used in the project

Type	Meaning
VM (Virtual Machine)	A full computer created in software on a bigger physical machine. You install an OS and programs on it.
Container	A lighter package that holds one app and everything it needs. Many containers can share one OS. Example: Docker.
Database	A managed service that stores data, like PostgreSQL or Redis.
Storage	A place to keep files, like Amazon S3 (images, backups).
Function (serverless)	A small piece of code that runs only when called; you pay per run. Example: resize an image when it is uploaded.

2.4 The four metrics

Metric	What it means	Why it matters
CPU %	How busy the processor is. 100% = fully busy.	At very high CPU, everything becomes slow.
Memory % (RAM)	How much of the working memory is used.	When RAM is full, programs crash or become very slow.
Disk %	How much of the storage space is used (space, not activity).	When the disk is full, the server cannot save anything.
Network in/out (KB/s)	Data received and sent per second.	Shows traffic; a sudden jump can mean heavy load or an attack.

Note: Task Manager's "Disk" column shows disk *activity*. CloudPulse shows disk *space used*. That is why they look different, and both are correct.

2.5 Monitoring words

Monitoring

Measuring the health of systems again and again, automatically.

Metric

One measured number, like CPU = 45%.

Real-time / live

Data shown almost at the moment it happens (here, every 3 seconds).

Threshold

A limit. If the value crosses it, something is wrong. Example: 95%.

Alert

A warning message created when a value crosses a threshold.

Severity

How serious an alert is: INFO, WARNING or CRITICAL.

Status

The overall state of a resource: RUNNING, WARNING, CRITICAL or STOPPED.

Log

A line of text a program writes to record what it did, with time and level, e.g. `12:00:03 ERROR db: connection timeout`.

Uptime

How long a machine has been running since its last restart.

Dashboard

One screen showing the most important numbers and charts.

2.6 How status is decided

For each resource, CloudPulse takes the **highest** of CPU, memory and disk:

Highest value	Status	Colour
90% or more	CRITICAL	Red
75% to 89.9%	WARNING	Amber
below 75%	RUNNING	Green
switched off	STOPPED	Grey

```
double worst = Math.max(r.getCpu(), Math.max(r.getMemory(), r.getDisk()));  
if (worst >= 90) return CRITICAL;  
if (worst >= 75) return WARNING;  
return RUNNING;
```

How web applications work

3.1 Client and server

A web app has two sides. The **client** (the browser) shows the screen and takes clicks. The **server** (Spring Boot) keeps the data and does the work. They talk over the network.

- **Frontend** = the part the user sees (React, in the browser).
- **Backend** = the part running on the server (Java Spring Boot).
- **Full-stack** = a project with both, like CloudPulse.

3.2 HTTP, request and response

HTTP is the language browsers and servers use. The browser sends a **request**; the server sends back a **response**.

HTTP method	Used for	Example in CloudPulse
GET	Read data	<code>GET /api/resources</code> – list all servers
POST	Create / do an action	<code>POST /api/auth/login</code> , <code>POST /api/resources/5/action</code>
PUT	Update	<code>PUT /api/costs/budget</code>
DELETE	Delete	<code>DELETE /api/alerts/rules/3</code>

3.3 Status codes

Code	Meaning	When you see it here
200 OK	Success	Normal answers
400 Bad Request	The request is wrong	Unknown action, or trying to stop your own PC
401 Unauthorized	Not logged in / token missing or expired	Calling the API without a token
403 Forbidden	Logged in, but not allowed	A viewer trying to stop a server
404 Not Found	Does not exist	Resource id that is not in the database
409 Conflict	Cannot do it in the current state	Stopping a server that is already stopped
500	Error inside the server	A bug

3.4 JSON

JSON (JavaScript Object Notation) is the text format used to send data between frontend and backend. Example of one resource:

```
{ "id": 1, "name": "web-server-01", "provider": "AWS",  
  "status": "RUNNING", "cpu": 42.5, "memory": 61.3, "disk": 48.2 }
```

Spring Boot converts Java objects to JSON automatically (with a library called Jackson).

3.5 API and REST

API (Application Programming Interface)

A set of URLs that one program can call to talk to another program.

REST

A simple style of API: each URL is a "resource" (noun), and the HTTP method says what to do with it (verb).

Each request is independent (stateless).

Endpoint

One URL + method, for example `GET /api/alerts/active`.

3.6 URL, host, port, localhost

- `http://localhost:5173`: **localhost** means "this same computer". **5173** is the **port**, the door number of a program.
- The React dev server runs on port **5173**; Spring Boot runs on port **8080**; MySQL runs on port **3306**.

3.7 Proxy (how the two ports work together)

The browser opens only `localhost:5173`. The Vite dev server **forwards** every call that starts with `/api` or `/ws` to Spring Boot on 8080. This forwarding is called a **proxy**.

```
'/api': 'http://localhost:8080',  
'/ws': { target: 'ws://localhost:8080', ws: true }
```

This is why you saw the error `ws proxy ECONNREFUSED` once: Vite tried to forward to 8080, but the backend was not running yet. Start the backend first.

3.8 CORS

CORS (Cross-Origin Resource Sharing) is a browser safety rule: a page from one address may not call a server on another address unless the server allows it. The backend allows `http://localhost:5173` in `app.cors.allowed-origins`.

Java and Spring Boot (the backend)

4.1 Java, JDK and JVM

Java 21

The programming language of the backend. Version 21 is an LTS (Long-Term Support) version.

JDK

Java Development Kit: the tools to write and compile Java.

JVM

Java Virtual Machine: runs the compiled code (bytecode). This is why Java runs on Windows, Linux and Mac ("write once, run anywhere").

4.2 Maven and pom.xml

Maven is a build tool. In `pom.xml` you list the libraries (**dependencies**) you need, and Maven downloads them and builds the project. In IntelliJ, "Reload All Maven Projects" makes it read pom.xml again after you add a library.

Main dependencies: spring-boot-starter-web, -data-jpa, -security, -websocket, -mail, -validation; mysql-connector-j; jjwt (JWT); oshi-core; openpdf; lombok.

4.3 Spring and Spring Boot

Framework

Ready-made code that gives a structure; you fill in your own parts.

Spring

The most popular Java framework for building server applications.

Spring Boot

Spring with automatic setup ("auto-configuration") and a built-in web server (**Tomcat**). You run one main class and the server starts on port 8080.

Starter

A bundle dependency, e.g. `spring-boot-starter-mail` brings everything needed for email.

4.4 Beans and Dependency Injection (very important)

A **bean** is an object that Spring creates and manages for you. Classes marked `@Service`, `@Component`, `@RestController`, `@Repository` or `@Configuration` become beans.

Dependency Injection (DI) means: a class does not create the objects it needs with `new`; Spring gives (injects) them. In CloudPulse it is done through the constructor:

```
public MonitoringService(CloudResourceRepository resources,
                        SimulationService simulation,
                        AlertService alertService, ...) {
    this.resources = resources; // Spring passes these in
    ...
}
```

Easy example: A restaurant cook does not go to the market himself. The manager (Spring) brings the vegetables (dependencies) to the kitchen. The cook just cooks. This makes classes simpler and easier to test or replace.

IoC (Inversion of Control): the general idea that the framework, not your code, creates and connects objects. DI is how Spring does IoC.

4.5 Important annotations used in the project

An **annotation** is a label starting with `@` that gives extra information to Java/Spring.

Annotation	Meaning
@SpringBootApplication	On the main class <code>CloudMonitorApplication</code> ; starts everything.
@EnableScheduling	Turns on scheduled jobs (@Scheduled).
@RestController	A class that answers HTTP requests with JSON.
@RequestMapping("/api/alerts")	Base URL for all methods of a controller.
@GetMapping, @PostMapping, @PutMapping, @DeleteMapping	Connect a method to a URL and HTTP method.
@PathVariable, @RequestParam, @RequestBody	Read a value from the URL path ({id}), from ?range=1h, or from the JSON body.
@Service	A class with business logic.
@Component	A general Spring bean (e.g. DataSeeder).
@Configuration + @Bean	A class that creates settings objects, like the password encoder.
@Entity, @Table, @Id, @GeneratedValue, @Column, @Enumerated	Database mapping (see Chapter 6).
@Scheduled(fixedRate = 3000)	Run a method again every 3000 ms.
@Value("\${app.monitor.tick-ms}")	Read a value from application.properties.
@PreAuthorize("hasRole('ADMIN')")	Only admins may call this method.
@PostConstruct	Run this method once after the bean is created (used to check the Gmail setup).
@Getter @Setter @NoArgsConstructor (Lombok)	Lombok writes getters, setters and empty constructors for you, so classes are shorter.

4.6 application.properties

The settings file of Spring Boot (`backend/src/main/resources`). Important lines:

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/cloudmonitor?createDatabaseIfNotExist=true
spring.jpa.hibernate.ddl-auto=update
app.jwt.expiration-ms=28800000          # 8 hours
app.monitor.tick-ms=3000                # every 3 seconds
app.monitor.persist-every=5             # history every 5 ticks = 15 s
app.monitor.retention-hours=24
app.cost.monthly-budget=3000
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=${MAIL_USERNAME:}
app.mail.max-per-hour=20
```

`${MAIL_USERNAME:}` means "read the environment variable MAIL_USERNAME; if it is missing use empty".

`${DB_PASSWORD:root123}` means "use DB_PASSWORD, or root123 if not set".

4.7 Layered architecture



Layer	Job	Example
Controller	Receives the HTTP request, calls a service, returns JSON. No heavy logic.	ResourceController
Service	Business logic: calculations, rules, decisions.	AlertService, CostService
Repository	Reads and writes the database.	AlertRepository
Model / Entity	Java class that matches a table.	CloudResource
DTO	Data Transfer Object: a small class only for sending data in/out of the API.	LoginRequest, CostReport

Why layers? Each part has one job (Separation of Concerns). If tomorrow the data comes from real AWS, only one service changes.

Project structure: every file explained

5.1 Folders

```
cloud-monitor/
├── backend/                                (Spring Boot, Java)
│   ├── pom.xml
│   └── src/main/java/com/cecil/cloudmonitor/
│       ├── CloudMonitorApplication.java
│       ├── config/   controller/  dto/   model/
│       ├── repository/ security/  service/
│       └── src/main/resources/application.properties
└── frontend/                             (React)
    ├── package.json  vite.config.js
    └── src/ api/ components/ context/ hooks/ pages/ utils/
```

5.2 Backend classes

Package	Class	What it does
(root)	CloudMonitorApplication	Main class. Starts Spring Boot.
config	DataSeeder	On first start fills the empty database: 2 users, 4 alert rules, 11 resources and 24 hours of history (one point per minute).
	SecurityConfig	Security rules: which URLs are open, JWT filter, BCrypt encoder, CORS, stateless sessions.
	WebSocketConfig	Opens the <code>/ws</code> endpoint and the <code>/topic</code> message broker.
controller	AuthController	<code>POST /api/auth/login</code> , <code>GET /api/auth/me</code>
	OverviewController	<code>/api/overview</code> totals and <code>/api/overview/history</code>
	ResourceController	List, details, history, alerts of a resource; start/stop/restart (admin)
	AlertController	Alerts, acknowledge, acknowledge all, rules CRUD (admin)
	CostController	<code>/api/costs</code> , change budget (admin)
	EmailSettingsController	Email settings and test email (admin)
	LogController	<code>/api/logs</code> and <code>/api/logs/download</code>
	ReportController	summary.pdf, resources.csv, alerts.csv, costs.csv

service	MonitoringService	Heart of the app. Every 3 s collects metrics, saves, checks alerts, writes logs, pushes to browsers. Also start/stop and history charts.
	SimulationService	Creates realistic values for the 10 cloud resources.
	LocalMetricsService	Reads real values of your PC using OSHI.
	AlertService	Checks rules, creates alerts, cooldown, acknowledge.
	EmailAlertService	Sends HTML alert emails through Gmail in the background, 20/hour limit.
	CostService	Burn rate, monthly projection, budget, savings tips.
	LogService	Keeps last 5,000 log lines in memory and streams new ones.
	ReportService	Builds the PDF (OpenPDF) and CSV files.
security	JwtService	Creates and checks JWT tokens.
	JwtAuthFilter	Runs before every request, reads the token from the Authorization header, logs the user in for that request.
	AppUserDetailsService	Loads a user from the database for Spring Security.
	StompAuthInterceptor	Checks the token when the browser connects to the WebSocket.
model	CloudResource, MetricSnapshot, AlertRule, Alert, AppUser, AppSetting + enums	Entities (tables) and fixed lists of values.
repository	One per entity	Database access (Spring Data JPA).
dto	LoginRequest, AuthResponse, LiveUpdate, OverviewDto, MetricPoint, CostReport, LogEntry ...	Shapes of data sent through the API.

5.3 Enums

An **enum** is a fixed list of allowed values. It stops spelling mistakes.

```
enum Provider      { AWS, AZURE, GCP, LOCAL }
enum ResourceType  { VM, DATABASE, CONTAINER, STORAGE, FUNCTION, LOCAL_HOST }
enum ResourceStatus { RUNNING, WARNING, CRITICAL, STOPPED }
enum MetricType    { CPU, MEMORY, DISK, NETWORK_IN, NETWORK_OUT }
enum Severity      { INFO, WARNING, CRITICAL }
enum Role          { ADMIN, VIEWER }
```

5.4 Frontend files

Folder	Files	What they do
pages	Login, Dashboard, Resources, ResourceDetail, Alerts, Costs, Logs	One file per screen.

components	Layout (sidebar/nav), Gauge, MetricChart, Sparkline, Toasts (pop-ups), ResourceActions, ExportMenu, ResourceIcon, ui	Reusable pieces of the screen.
context	AuthContext, LiveContext, ThemeContext	Shared data for all pages: logged-in user, live WebSocket data, dark/light theme.
api	client.js	Axios setup: adds the JWT token to every call; file downloads.
hooks	useHistory.js	Loads chart history for a time range.
utils	format.js	Formats numbers, bytes, times.

The database

6.1 Basic words

Database

An organised store of data. CloudPulse's database is named `cloudmonitor`.

DBMS

The software that manages databases. Here: **MySQL 8**.

Relational database

Data is kept in **tables** (rows and columns) that can be linked.

Table / Row / Column

Table = like an Excel sheet. Row (record) = one item. Column (field) = one property.

Primary key

A column that uniquely identifies each row. Here every table has `id`, auto-numbered (`AUTO_INCREMENT`).

Foreign key / link

A column that points to a row in another table. Here `resource_id` in snapshots and alerts points to `cloud_resources.id`.

Index

A shortcut that makes searches faster, like the index of a book. `metric_snapshots` has an index on (`resource_id`, `timestamp`) because charts search by these.

SQL

Structured Query Language: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE`.

CRUD

Create, Read, Update, Delete – the four basic data operations.

6.2 The six tables

Table	Main columns	Stores
<code>cloud_resources</code>	<code>id</code> , <code>name</code> , <code>type</code> , <code>provider</code> , <code>region</code> , <code>instance_type</code> , <code>ip_address</code> , <code>os</code> , <code>status</code> , <code>cpu</code> , <code>memory</code> , <code>disk</code> , <code>network_in</code> , <code>network_out</code> , <code>hourly_cost</code> , <code>started_at</code> , <code>last_updated</code>	Each server and its latest values
<code>metric_snapshots</code>	<code>id</code> , <code>resource_id</code> , <code>timestamp</code> , <code>cpu</code> , <code>memory</code> , <code>disk</code> , <code>network_in</code> , <code>network_out</code>	History points for charts (every 15 s, kept 24 h)
<code>alert_rules</code>	<code>id</code> , <code>name</code> , <code>metric</code> , <code>operator</code> (> or <), <code>threshold</code> , <code>resource_id</code> (empty = all), <code>severity</code> , <code>enabled</code>	Alert conditions
<code>alerts</code>	<code>id</code> , <code>rule_id</code> , <code>rule_name</code> , <code>resource_id</code> , <code>resource_name</code> , <code>metric</code> , <code>metric_value</code> , <code>threshold</code> , <code>severity</code> , <code>message</code> , <code>created_at</code> , <code>acknowledged</code> , <code>acknowledged_by</code>	Every alert raised
<code>app_users</code>	<code>id</code> , <code>username</code> (unique), <code>password</code> (BCrypt hash), <code>display_name</code> , <code>role</code>	Logins

6.3 Relationships

- One **resource** → many **snapshots** (one-to-many)
- One **resource** → many **alerts**
- One **rule** → many **alerts**
- A rule may target one resource, or all (resource_id empty)

The project links tables by storing the id (`resourceId`) as a simple column. Alerts also copy the resource name, so an alert still makes sense later in reports.

6.4 Default alert rules (created on first start)

Rule	Condition	Severity
High CPU	CPU > 85%	WARNING
CPU critical	CPU > 95%	CRITICAL
High memory	Memory > 88%	WARNING
Disk almost full	Disk > 90%	CRITICAL

6.5 JPA, Hibernate and ORM

ORM (Object-Relational Mapping)

Converting Java objects to table rows and back automatically.

JPA (Jakarta Persistence API)

The standard Java rules for ORM (the annotations).

Hibernate

The library that actually does the ORM work and writes the SQL.

Spring Data JPA

Makes repositories: you write an interface and Spring writes the code.

JDBC

Low-level Java way to talk to a database. Hibernate uses it underneath, with the MySQL driver (mysql-connector-j).

```

@Entity
@Table(name = "alerts")
public class Alert {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Enumerated(EnumType.STRING)           // store "CRITICAL", not 2
    private Severity severity;
    @Column(name = "metric_value")         // "value" is a reserved word
    private double value;
    ...
}

```

Repository example. Spring creates the SQL from the method *name*:

```

public interface AlertRepository extends JpaRepository<Alert, Long> {
    List<Alert> findByAcknowledgedFalseOrderByCreatedAtDesc();
    long countByAcknowledgedFalse();
}
// = SELECT * FROM alerts WHERE acknowledged = false ORDER BY created_at DESC

```

`JpaRepository` already gives `save()`, `findAll()`, `findById()`, `deleteById()`, `count()`.

6.6 ddl-auto=update

DDL (Data Definition Language) = commands that create tables. With `ddl-auto=update`, Hibernate creates the tables from the entity classes on start, and adds new columns if a class changes. It never deletes your data.

6.7 H2 vs MySQL

	H2	MySQL
Where data lives	In memory (RAM)	On disk
After restart	Everything lost	Data kept
Setup	Nothing to install	Install MySQL server
Used for	Quick start, testing	Real use

The project started on H2, then moved to MySQL. To go back, comment the MySQL lines and un-comment the H2 lines in application.properties.

6.8 Keeping the database small

- Latest values are **updated** in cloud_resources every 3 s (11 rows, not new rows).
- History is **added** only every 15 s.
- Every 10 minutes a cleanup job deletes history older than 24 hours.
- For long ranges the server **averages** history into at most 90 points ("bucketing"), so charts stay fast.

Live updates: the 3-second loop and WebSocket

7.1 Polling vs pushing

	Polling (normal HTTP)	Pushing (WebSocket)
How	Browser asks "anything new?" again and again	Connection stays open; server sends new data itself
Speed	Delay until the next ask	Instant
Waste	Many empty requests	One connection, only real messages

Example: Polling is like calling a shop every minute to ask "is my order ready?". WebSocket is like the shop calling you when it is ready.

7.2 WebSocket, STOMP, topic, broker

WebSocket

A two-way connection between browser and server that stays open. Address: `ws://localhost:8080/ws`.

STOMP

Simple Text Oriented Messaging Protocol. A simple message format on top of WebSocket with commands like CONNECT, SUBSCRIBE, SEND.

Topic

A named channel. Anyone subscribed to it gets every message sent to it.

Message broker

The part that delivers messages to all subscribers. Spring's simple in-memory broker handles `/topic`.

Publish / Subscribe (pub-sub)

The sender publishes to a topic without knowing who is listening; listeners subscribe. Like a YouTube channel and its subscribers.

Topic	What is sent	How often
<code>/topic/live</code>	Overview totals + all 11 resources	Every 3 s
<code>/topic/alerts</code>	Each new alert	When it fires
<code>/topic/logs</code>	Each new log line	As they are written

The server sends with `SimpMessagingTemplate`:

```
messaging.convertAndSend("/topic/live", new LiveUpdate(overview, resources));
```

The browser (LiveContext.jsx, library `@stomp/stompjs`) subscribes:

```
client.subscribe('/topic/live', (msg) => {
  const { overview, resources } = JSON.parse(msg.body)
  applyUpdate(overview, resources) // React redraws the screen
})
```

If the connection drops, the client **reconnects every 3 seconds** automatically (`reconnectDelay: 3000`).
Heartbeats every 10 s check that the connection is still alive.

7.3 The 3-second loop step by step (MonitoringService.collect)

```
@Scheduled(fixedRateString = "${app.monitor.tick-ms:3000}", initialDelay = 2000)
public void collect() { ... }
```

1. **Load** all resources from MySQL.
2. For each resource:
 - if STOPPED → set CPU, memory and network to 0;
 - if it is your PC → `LocalMetricsService.update()` (real values from OSHI);
 - otherwise → `SimulationService.update()` (simulated values);
 - work out the status (RUNNING / WARNING / CRITICAL).
3. **Save** all resources (`saveAll`).
4. Every 5th tick (= 15 s) **save history** points for running resources.
5. **Check alert rules** (`alertService.evaluate`).
6. **Write log lines** (`logs.generate`).
7. **Push** to `/topic/live`.

Tick

One run of the loop.

fixedRate

Start a new run every 3000 ms, measured from the start of the previous run.

initialDelay

Wait 2 s after start-up before the first run.

synchronized (lock)

Makes sure the loop and a Start/Stop click do not change the same data at the same time (thread safety).

Thread

A separate line of work inside a program. The scheduler, web requests and email sending each run on their own threads.

7.4 Start-up: before the first tick

`DataSeeder` (a `CommandLineRunner`, which runs once when the app starts) fills an empty database, and then calls `monitoring.markReady()`. Until then, the loop does nothing. It also fills the log buffer with the last 10 minutes of lines so the Logs page is not empty.

Monitoring: simulated servers and your real PC

8.1 Why simulation?

Connecting real AWS, Azure and Google Cloud accounts needs paid subscriptions and credit cards. So 10 resources are **simulated**: code creates values that behave like real servers. Because all of it is inside **one class** (SimulationService), it can later be replaced by real cloud APIs without changing anything else.

8.2 How the simulation makes realistic numbers

1. **Personality (profile)**: each resource gets a base CPU and memory by its type. Databases use more memory; storage uses very little CPU; the ML/GPU server is always busy (CPU ~68%). The random seed comes from the resource id, so each resource always has the same personality.
2. **Daily pattern**: load goes up and down like a real day (a sine wave), busiest around 3 PM, quietest at night:

```
factor = 1 + 0.30 × sin(2π × (minuteOfDay - 540) / 1440)
```

So load is 30% above normal in the afternoon and 30% below at night.

3. **Noise**: small random changes (Gaussian/normal distribution) so lines are not perfectly smooth.
4. **Smooth movement**: the value moves 35% of the way to its target each tick, so it does not jump.
5. **Spikes**: each tick there is a **0.15%** chance that a resource gets a load spike: CPU goes to 86–99% for 30–90 seconds, memory +12%, network ×2.5. With 10 resources that is about one spike every 3 minutes, which creates realistic alerts.
6. **Disk**: slowly fills a tiny bit each tick; at 96% a "cleanup" frees space back to 55–65%.
7. **Clamp**: values are kept between 0 and 100.

History at first start: the seeder creates the past 24 hours (one point per minute) with the same daily pattern, so charts are full from the first minute. Your PC has no fake history: only real readings.

8.3 OSHI: real data from your PC

OSHI (Operating System and Hardware Information) is a free Java library that reads hardware information from Windows, Linux or Mac without installing anything else.

Value	How it is calculated
CPU %	OSHI compares two readings of CPU "ticks" (busy vs idle time) taken 3 s apart: <code>getSystemCpuLoadBetweenTicks</code>
Memory %	$(\text{total} - \text{available}) \div \text{total} \times 100$
Disk %	$(\text{total space} - \text{free space}) \div \text{total} \times 100$ (space used, not activity)
Network	Bytes received/sent since last tick, summed over all adapters, in KB/s

```
Uptime    startedAt = now - os.getSystemUptime()
```

You verified these against Windows Task Manager: memory 88.9% vs 89%, and the same uptime.

Why Task Manager uptime was different at first: the app first showed "time since the app started". It was fixed to use the real Windows uptime from OSHI. (Windows "Fast Startup" can also make uptime longer than expected, because shutdown is really a hibernate.)

Your PC cannot be stopped from the dashboard (the server returns 400 Bad Request). Only "refresh" is allowed for it, which is safer.

8.4 Charts and time ranges

- The dashboard keeps the last **100 live points** (≈5 minutes) in the browser.
- History charts ask the server for **15 min, 1 h, 6 h or 24 h**: `GET /api/resources/{id}/metrics?range=24h`.
- The server groups history into at most **90 buckets** and averages each, so a 24-hour chart has 90 points instead of 5,760.

8.5 Start, stop, restart

Action	What happens
STOP	Status STOPPED, CPU/memory/network 0, no cost, no history, no alerts. An event "stopped by admin" (WARNING) is recorded.
START	Status RUNNING, start time = now, values reset to half of normal. INFO event.
RESTART	Same as start: clears any spike.

Only admins can do these (`@PreAuthorize("hasRole('ADMIN')")`). Each action is saved as a "System event" alert with the user's name, which works as an **audit trail** (record of who did what).

Alerts and email notifications

9.1 Alert rules

A rule has: a **name**, a **metric** (CPU, MEMORY, DISK, NETWORK_IN, NETWORK_OUT), an **operator** (> or <), a **threshold**, a **severity**, an optional single **resource** (empty = all resources) and an **enabled** on/off switch. Admins can create, edit, switch off and delete rules on the Alerts page.

9.2 How rules are checked (AlertService.evaluate)

1. Take all **enabled** rules.
2. For each rule and each **running** resource (if the rule is for one resource, skip the others):
3. Read the value. Is `value > threshold` (or <)? If not, continue.
4. **Cooldown**: if the same rule + same resource already fired in the last **2 minutes**, skip it. (Kept in a map with the key `"ruleId:resourceId"`.)
5. Create an **Alert** with a message like `"web-server-01 CPU is 97.2% (> 95)"` and save it.
6. **Publish** it.

9.3 What happens when an alert is published

Pop-up in every browser
(/topic/alerts)

Saved in MySQL
(Active alerts)

Written to logs
(ERROR/WARN/INFO)

Email if CRITICAL

User acknowledges

- **Toast / pop-up**: a small message in the corner of the screen (max 4 at a time).
- **Active alerts** = alerts not yet acknowledged. The count is shown on the dashboard.
- **Acknowledge** = "I have seen it and I am handling it". One by one or "Acknowledge all"; the username is saved (acknowledged_by).
- Severity → log level: CRITICAL → ERROR, WARNING → WARN, INFO → INFO.

9.4 Email: the words

SMTP

Simple Mail Transfer Protocol: the standard way programs send email.

smtp.gmail.com : 587

Gmail's sending server and port. Port 587 uses **STARTTLS**: the connection starts normal and then switches to encrypted (TLS).

TLS

Transport Layer Security: encryption so nobody on the network can read the password or email.

Gmail App password

A 16-letter password made in Google Account → Security (needs 2-Step Verification). It is used by apps instead of your real password and can be removed at any time.

Environment variable

A named setting given to a program when it starts, outside the code. Here `MAIL_USERNAME` and `MAIL_PASSWORD` are set in IntelliJ's Run Configuration. So the password is **never in the code or on GitHub**.

JavaMailSender

The Spring class that sends emails (from spring-boot-starter-mail).

HTML email

An email with design (colours, a table, a button "Open dashboard").

9.5 How email sending works (EmailAlertService)

1. At start (`@PostConstruct`) it checks the Gmail setup and prints *"Email alerts: ready ... (password length 16)"*. It never prints the password. It also removes stray spaces (a real bug you had).
2. For every alert, `onAlert()` checks: Is email set up and switched on? Is severity at least the minimum (default CRITICAL)? Are there recipients (max 5)? Is the limit of **20 emails per hour** reached?
3. If all OK, the email is handed to a **background thread** (an executor). So the 3-second loop never waits for Gmail.
4. Admins can change recipients, turn email on/off and send a **test email** from the Email tab. Settings are stored in `app_settings`.

Spam guards: (1) the same rule on the same server fires at most once every 2 minutes; (2) at most 20 emails per hour; (3) only CRITICAL alerts are emailed by default.

Costs and budget

10.1 Words

Pay-as-you-go

Cloud servers are billed per hour while they run. A stopped server costs nothing (here).

Burn rate

Money spent per hour right now = sum of hourly prices of all **running** resources.

Projected monthly cost

Burn rate $\times$ 730.

730 hours

Cloud providers count a month as $365 \times 24 \div 12 = 730$ hours.

Budget

The planned monthly spending limit. Default \$3,000 (`app.cost.monthly-budget`); admins can change it; saved in `app_settings`.

10.2 Worked example

```
Running: 0.0448 + 0.0448 + 0.0494 + 0.152 + 0.031 + 0.155
         + 0.006 + 0.10 + 3.06           = $3.643  $\approx$  $3.64 per hour (backup-server stopped)
Month:   3.643  $\times$  730                    $\approx$  $2,659
Budget:  2,659  $\div$  3,000  $\times$  100           $\approx$  89% of budget
```

The GPU server (\$3.06/h) alone is about 84% of the bill.

10.3 What the Costs page shows

- Burn rate, projected month and a bar showing how much of the budget is used.
- Cost by provider and by type; cost of each resource and its share %.
- Spending per hour for the last 24 hours (from history: a resource is charged for an hour if it has history in that hour).
- Savings already made by stopped servers.
- **Savings tips:**
 - **SCHEDULE:** if one resource is 40% or more of the bill $\rightarrow$ "run it only in working hours"; stopping 12 hours a night halves its cost.
 - **DOWNSIZE:** a VM/database/container whose average CPU in 24 h is below 25% (and peak below 80%) is too big $\rightarrow$ use a smaller size, saving about half.

Your PC costs \$0 and is left out of the cost totals.

Logs

11.1 Words

Log line

Time + level + source + message. Example: `14:02:11 WARN nginx GET /api/cart 200 812ms`.

Log level

DEBUG (details for developers) < INFO (normal) < WARN (warning) < ERROR (problem).

Buffer

A fixed-size store in memory. CloudPulse keeps the newest **5,000** lines; when full, the oldest is removed.

Streaming

Sending each new line to the browser as it is created (via `/topic/logs`).

11.2 Where the lines come from

- Simulated servers have no real log files, so LogService writes **believable lines** for each type: web requests for web servers, queries and checkpoints for databases, uploads for storage, training progress for the ML worker. When a server is under load, its lines become slower and more serious (WARN/ERROR), so logs match the charts and alerts.
- Your PC gets **real** lines from its actual CPU and memory, about every 30 seconds.
- CloudPulse writes its own system lines too, like every alert.

11.3 Logs page features

Terminal-style view; filter by server and minimum level; text search; pause/resume; highlight of new lines; download as a .log text file (`/api/logs/download`). A "Logs" button on a resource's page opens logs filtered for it.

Limitation: logs live in memory, so they are lost on restart (the last 10 minutes are re-created at start). Saving them in a database is future work.

Reports and export

PDF

Portable Document Format: looks the same on every device. Made in Java with the **OpenPDF** library (tables, colours, fonts).

CSV

Comma-Separated Values: plain text table, one row per line, opens in Excel.

Content-Disposition: attachment

An HTTP header that tells the browser "download this as a file with this name" instead of showing it.

Blob

Binary data in the browser. client.js downloads the file as a blob (with the JWT token) and then saves it.

Endpoint	File
/api/reports/summary.pdf	2-page report: overview, all resources, recent alerts, costs
/api/reports/resources.csv	All resources and their values
/api/reports/alerts.csv	Alerts
/api/reports/costs.csv	Costs per resource

The **ExportMenu** component on the frontend shows these options.

Security

13.1 Authentication vs Authorization

	Authentication	Authorization
Question	Who are you?	What are you allowed to do?
In CloudPulse	Username + password login, then JWT token	Roles ADMIN / VIEWER, @PreAuthorize
Failure code	401 Unauthorized	403 Forbidden

Example: At a college gate, the ID card check is authentication. Being allowed into the staff room or not is authorization.

13.2 Password hashing with BCrypt

Hashing

Turning text into a fixed scrambled string that **cannot be turned back**. Not the same as encryption (which can be decrypted).

Salt

Random extra data added before hashing, so two users with the same password get different hashes.

BCrypt

A password hashing method that is **slow on purpose** and salted. Slow means guessing billions of passwords is impractical.

```
admin123 → $2a$10$N9qo8uL0ickgx2ZMRZoMye... (stored in app_users.password)
```

At login, Spring hashes the typed password with the same salt and compares the results. The real password is never stored. (`$2a$` = BCrypt, `10` = cost/strength.)

13.3 JWT (JSON Web Token)

A JWT is a signed "pass" the server gives after a correct login. It has three parts separated by dots:

```
xxxxx.yyyyy.zzzzz
Header . Payload . Signature
```

- **Header:** the signing method (HMAC-SHA).
- **Payload (claims):** `sub` = username, `role`, `iat` = issued at, `exp` = expiry (8 hours later).
- **Signature:** made with a secret key (`app.jwt.secret`) that only the server knows. If anyone changes the payload (e.g. role to ADMIN), the signature no longer matches and the token is rejected.

The payload is only Base64-encoded, **not encrypted**. Anyone can read it, but nobody can change it without the secret. So never put passwords in a JWT.

13.4 The login flow

1. Browser sends `POST /api/auth/login` with `{username, password}`.
2. AuthController asks Spring's **AuthenticationManager** to check it (BCrypt compare). Wrong → 401 "Invalid username or password".
3. Correct → `JwtService.generateToken()` returns a token + username + display name + role.
4. React saves the token in **localStorage** (browser storage).
5. From now on, `client.js` adds the header `Authorization: Bearer <token>` to every API call (an Axios **interceptor**).
6. If any call returns 401 (token expired), the app clears the token and goes back to the login page.

13.5 What happens on every request

1. **JwtAuthFilter** runs first. It reads the `Authorization` header, checks the signature and expiry, loads the user and puts them in the **SecurityContext** (Spring's "who is the current user" box).
2. **SecurityConfig** rules: `/api/auth/login`, `/ws/**` and `/error` are open; every other URL needs login.
3. `@PreAuthorize("hasRole('ADMIN')")` on admin methods blocks viewers with 403.
4. WebSocket: **StompAuthInterceptor** checks the token in the STOMP CONNECT message.

13.6 Other security words

Stateless

The server does not keep login sessions in memory; every request carries its own token. Easy to scale to many servers.

Session

The old way: the server remembers who is logged in using a cookie. Not used here.

CSRF

Cross-Site Request Forgery: an attack that misuses login cookies. Disabled here because the app does not use cookies (it uses a header token).

Filter chain

A list of security checks every request passes through, one after another.

Role-Based Access Control (RBAC)

Permissions given by role, not per person.

Server-side check

Hiding a button is not security. The server itself must refuse. CloudPulse does both.

Secrets management

Passwords and keys stay out of code: Gmail password in environment variables, DB password can come from `DB_PASSWORD`.

13.7 Permissions table

Action	Admin	Viewer
See dashboards, resources, logs, costs	Yes	Yes
Acknowledge alerts, export reports	Yes	Yes
Start, stop, restart servers	Yes	No
Create, edit, delete alert rules	Yes	No
Change budget and email settings, send test email	Yes	No

13.8 Honest weak points (and the fix)

- Runs on HTTP locally. In production use **HTTPS** so tokens cannot be read on the network.
- Token in localStorage can be read by malicious scripts (XSS). React escapes text by default, which helps.
- The JWT secret and the MySQL password are in application.properties. Keep the GitHub repo **private**, or move them to environment variables before making it public.
- Demo passwords (admin123) must be changed for real use.

The frontend (React)

14.1 Tools and libraries

Name	What it is
JavaScript	The programming language of browsers.
Node.js	Runs JavaScript outside the browser; needed for the development tools.
npm	Node Package Manager. <code>npm install</code> downloads libraries listed in <code>package.json</code> into <code>node_modules</code> . <code>npm run dev</code> starts the dev server.
React 19	Library to build the screen from components .
JSX	HTML-like code written inside JavaScript (.jsx files).
Vite	Very fast dev server and build tool. Runs on port 5173; reloads the page when you save a file; also the proxy to 8080.
Tailwind CSS	Styling with small ready classes like <code>p-4 rounded-xl bg-white</code> instead of writing CSS files.
Recharts	React charts library (line and area charts).
React Router	Switches pages (/, /resources, /alerts, /costs, /logs) without reloading.
Axios	Makes HTTP calls to the API.
@stomp/stompjs	WebSocket + STOMP client.
framer-motion	Animations.
lucide-react	Icons.

14.2 Core React ideas

Component

A function that returns a piece of screen, e.g. `<Gauge value={72} />`. Components are reused.

Props

Inputs given to a component (like function parameters), e.g. `value={72}`.

State (useState)

Data that can change. When state changes, React redraws only what changed.

useEffect

Runs code after the screen is drawn, e.g. open the WebSocket once, load data.

Context

A way to share data with all pages without passing props everywhere. LiveContext shares live data; AuthContext shares the logged-in user; ThemeContext shares dark/light mode.

Hook

Functions starting with `use` (useState, useEffect, useHistory) that add features to components.

Virtual DOM

React keeps a copy of the page in memory, compares it with the new one, and changes only the differences. That is why live updates are smooth.

SPA (Single Page Application)

One HTML page; JavaScript changes the content when you move between pages.

14.3 How the live screen works (LiveContext)

1. On start, it loads a first snapshot with REST (`/api/overview` and `/api/resources`) so the screen is not empty.
2. Opens the WebSocket with the token and subscribes to the three topics.
3. On each `/topic/live` message: updates resources, overview and adds a point to the rolling chart data (max 100 points).
4. On each alert: shows a toast and tells any listening page (e.g. Alerts page refreshes).
5. On each log line: sends it to the Logs page if it is open.
6. A "paused" switch lets the user freeze the live charts.

14.4 Pages

Page	Shows
Login	Username/password form.
Dashboard	Totals (running/warning/critical/stopped), average CPU/memory/disk, active alerts, cost per hour, live charts, provider breakdown.
Resources	Table/cards of all 11 resources with status, sparklines and actions.
Resource detail	Gauges, history charts (15m/1h/6h/24h), details (region, IP, OS, uptime), its alerts, Logs button, start/stop.
Alerts	Active / history, acknowledge, rules editor, Email tab.
Costs	Burn rate, projection, budget, breakdowns, savings tips.
Logs	Live log stream with filters, search, pause, download.

The design is **responsive**: it works on phone screens too.

Tools: IntelliJ, Git and GitHub

IDE

Integrated Development Environment. IntelliJ IDEA is used for the Java backend.

Run Configuration

IntelliJ settings for how to start the app: main class and **environment variables** (MAIL_USERNAME, MAIL_PASSWORD). "Store as project file" is unticked, so the password is not saved into the project.

Reload All from Disk

File → Reload All from Disk: makes IntelliJ read files that changed outside it.

Git

A version control system: it records snapshots (**commits**) of your code so you can see history and go back.

Commit

A saved snapshot with a message, e.g. "Add live logs viewer". Saved on your PC only.

Push

Sends your commits to GitHub.

Repository (repo)

The project folder with its whole history. Yours is **private** on GitHub.

GitHub

A website that stores Git repositories online.

.gitignore

A list of files Git must not save, like node_modules and target.

How to run the project

1. **MySQL** must be running (Windows service, usually named MySQL80). Database is created automatically.
2. **Backend:** open `backend` in IntelliJ and run `CloudMonitorApplication`. Wait for "Started CloudMonitorApplication" and "Email alerts: ready".
3. **Frontend:** in a terminal: `cd frontend`, then `npm install` (first time only), then `npm run dev`.
4. Open `http://localhost:5173` and log in as `admin / admin123`.

Order matters: MySQL → backend → frontend. If the frontend shows connection errors, the backend is not running yet.

Program	Port
React (Vite)	5173
Spring Boot	8080
MySQL	3306
Gmail SMTP	587

Problems faced and how they were solved

Examiners love this part. It shows you really built it.

Problem	Cause	Fix
Tables were not created	Column name "value" is a reserved SQL word	<code>@Column(name = "metric_value")</code>
Data lost on every restart	H2 is an in-memory database	Moved to MySQL
MySQL "Access denied for root"	Root password unknown	Reset with an init file (<code>ALTER USER</code>) and set root123
Startup crash: H2ConsoleProperties bean not found	An H2-only security rule (PathRequest.toH2Console) remained after switching to MySQL	Replaced it with a plain path rule <code>antMatcher("/h2-console/**")</code>
Over 150 alerts in 15 minutes	Spike chance too high (0.6% per tick)	Reduced to 0.15% + 2-minute cooldown
Email tab said "Not set up"	A hidden space in the environment variable name (" MAIL_USERNAME")	Code trims names and removes spaces from the App password; logs a clear message
Uptime did not match Task Manager	App showed time since app start	Use OSHI's real system uptime
Disk value different from Task Manager	Task Manager shows disk activity; app shows space used	Explained and labelled "Disk used"
<code>ws proxy ECONNREFUSED</code> in Vite	Frontend started before backend	Start backend first

Limitations and future scope

Limitation today	Future improvement
10 cloud servers are simulated	Use AWS CloudWatch, Azure Monitor and Google Cloud Monitoring APIs
Logs kept in memory only	Store logs in a database or Elasticsearch
Fixed hourly prices	Read real prices from cloud billing APIs
Manual testing only	JUnit and Mockito automated tests
Runs on one PC	Docker containers and cloud deployment with HTTPS
Alerts only when a limit is crossed	Prediction: "disk will be full in 3 days" from the trend
Email only	SMS, Slack or Telegram alerts

Glossary A-Z

Acknowledge

Mark an alert as seen/handled.

Alert

Warning created when a rule's limit is crossed.

Annotation

A `@Label` giving extra info to Java/Spring.

API

URLs programs use to talk to each other.

App password

16-letter Gmail password for apps.

Audit trail

Record of who did what and when.

Authentication

Checking who you are.

Authorization

Checking what you may do.

Backend

Server-side part (Spring Boot).

BCrypt

Slow, salted password hashing.

Bean

Object created and managed by Spring.

Broker

Delivers messages to topic subscribers.

Bucketing

Grouping many points into time slots and averaging.

Burn rate

Cost per hour of running resources.

Clamp

Keep a number inside a range (0-100).

Commit

Saved snapshot of code in Git.

Component

Reusable piece of React screen.

Container

Lightweight package for one app.

Context (React)

Shared data for all components.

Cooldown

Waiting time before the same alert can fire again (2 min).

CORS

Browser rule about calling other addresses.

CPU

Processor; CPU % = how busy it is.

CRUD

Create, Read, Update, Delete.

CSV

Text table file for Excel.

Dependency

A library your project needs.

Dependency Injection

Spring gives objects their needed objects.

DTO

Small class for sending data in/out.

Endpoint

One API URL + method.

Entity

Java class mapped to a table.

Enum

Fixed list of allowed values.

Environment variable

Setting given to a program from outside the code.

Executor

Runs tasks on background threads.

Frontend

Part the user sees (React).

H2

In-memory Java database.

Hash

One-way scrambled value.

Hibernate

ORM library that writes SQL for you.

Hook

React function starting with "use".

HTTP / HTTPS

Web protocol / its encrypted version.

Index

Database shortcut for faster search.

Interceptor

Code that runs on every request/message (Axios adds token; STOMP checks token).

JPA

Java standard for ORM.

JSON

Text data format between frontend and backend.

JVM

Runs Java bytecode.

JWT

Signed login token (header.payload.signature).

Localhost

This same computer.

Lombok

Writes getters/setters automatically.

Maven

Java build and library tool (pom.xml).

Metric

One measured number.

MySQL

Relational database server.

npm

JavaScript package manager.

ORM

Objects ↔ tables mapping.

OSHI

Java library that reads real hardware data.

Port

Door number of a program (8080, 5173, 3306).

Primary key

Unique id of a row.

Proxy

Forwards requests to another server.

Pub-sub

Publish to topics; subscribers receive.

React

UI library built from components.

Real-time

Shown almost immediately.

Region

Location of a cloud data centre.

Repository (JPA)

Interface that reads/writes a table.

REST

API style: URLs + HTTP methods.

Role

ADMIN or VIEWER.

Salt

Random data added before hashing.

Scheduler

Runs a method at fixed times (@Scheduled).

Seeder

Fills the database with starting data.

Service

Class with business logic.

Severity

INFO / WARNING / CRITICAL.

Simulation

Code that creates realistic fake values.

SMTP

Protocol for sending email.

SPA

Single-page web application.

Spring Boot

Java framework with auto-setup and built-in server.

SQL

Language for databases.

Stateless

Server keeps no session; token on each request.

Status code

Number in a response (200, 401, 403, 404...).

STOMP

Message format on top of WebSocket.

Tailwind

CSS with small utility classes.

Thread

Separate line of work inside a program.

Threshold

Limit value in a rule.

Tick

One 3-second run of the monitoring loop.

TLS

Encryption for network connections.

Toast

Small pop-up message.

Topic

Named message channel (/topic/live).

Uptime

Time since last restart.

Vite

Fast React dev server and build tool.

VM

Virtual machine: a computer made in software.

WebSocket

Always-open two-way browser-server connection.

Quick revision: 25 points to remember

1. CloudPulse = live cloud monitoring dashboard: health, alerts, email, costs, logs, reports.
2. Stack: Java 21 + Spring Boot 3.5, React 19 + Vite, MySQL 8.
3. 11 resources: 10 simulated (AWS, Azure, GCP) + 1 real PC through OSHI.
4. Three tiers: browser → Spring Boot → MySQL; plus OSHI and Gmail SMTP.
5. Layers: Controller → Service → Repository → Entity.
6. MonitoringService runs every 3 s with @Scheduled.
7. Loop: read values → save → history every 15 s → check rules → logs → push.
8. WebSocket + STOMP topics: /topic/live, /topic/alerts, /topic/logs.
9. Six tables: cloud_resources, metric_snapshots, alert_rules, alerts, app_users, app_settings.
10. Hibernate creates tables from @Entity classes (ddl-auto=update).
11. History kept 24 h; cleanup every 10 min; charts averaged into max 90 points.
12. Status: ≥90% critical, ≥75% warning, else running.
13. Simulation: type-based base + daily sine curve (peak 3 PM) + noise + 0.15% spikes.
14. Default rules: CPU >85 warning, CPU >95 critical, memory >88 warning, disk >90 critical.
15. Alert flow: pop-up → saved → logged → email if critical → acknowledge.
16. Spam guards: 2-min cooldown, 20 emails/hour, max 5 recipients, critical only.
17. Gmail via SMTP 587 with STARTTLS and an App password in environment variables.
18. Email sent on a background thread so monitoring never waits.
19. Cost: burn rate × 730 = projected month; budget \$3,000; SCHEDULE and DOWNSIZE tips.
20. Logs: 5,000 lines in memory, levels DEBUG/INFO/WARN/ERROR, live stream, download.
21. Reports: PDF with OpenPDF, CSV for Excel.
22. Passwords: BCrypt hashes. Login: JWT valid 8 hours in the Authorization header.
23. Roles: ADMIN changes things, VIEWER only watches; checked on the server with @PreAuthorize (403).
24. Bugs fixed: reserved word "value", H2 rule after MySQL, too many alerts, space in env variable.
25. Future: real cloud APIs, logs in DB, prediction, JUnit, Docker, HTTPS.